

Homework 2: Nondeterminism and Pumping Lemma

Due:02/05/2026

This assignment is due on **11:59 PM EST, Thursday, February 05, 2026**. You may turn it in up to 48 hours late, but assignments turned in by the deadline receive 3% extra credit. Additionally, late submission means late feedback, which means less time to study before an exam.

You should submit a typeset or *neatly* written pdf on Gradescope. The grading TA should not have to struggle to read what you've written; if your handwriting is hard to decipher, you will be asked to typeset your future assignments.

Enter your work using the `answer` environment we have provided for you. The default color is teal, but you can choose to change it to any of red, brown, magenta, violet, blue, darkgray, or black like so:

```
\begin{answer}[<COLOR>]
  <YOUR WORK GOES HERE>
\end{answer}
```

You may collaborate with other students, but any written work should be your own.

1. Nondeterminism (4 points)

For the following proofs, you must provide a construction of a finite automaton by describing its components to show that the language is regular.

(2 points - *Section A*) For any regular language L , prove that $L' = \{w \text{ where all } a\text{'s are removed} \mid w \in L, a \in \Sigma\}$ is regular.

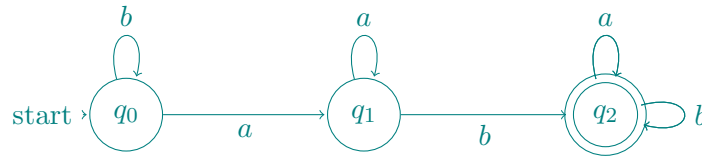
Answer:

(2 points - *Both*) For any regular language L , prove that $L' = \{xay \mid xy \in L, a \in \Sigma, x, y \in \Sigma^*\}$ is regular. Notice that we form L' by taking the strings in L , and adding a single symbol anywhere in the string, even possibly at the beginning or end.

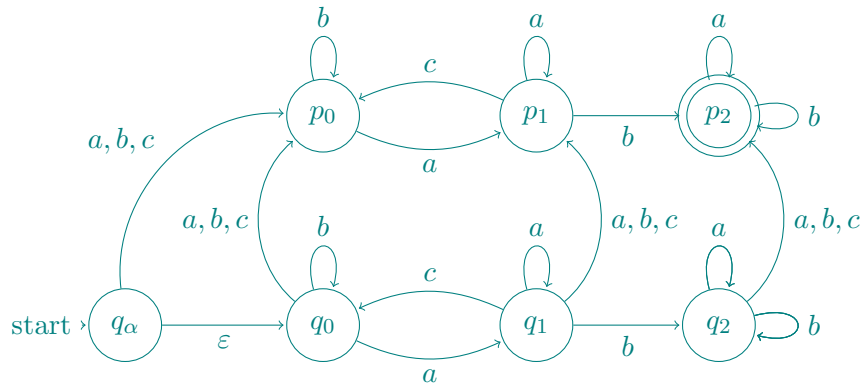
Answer: First let us denote the addition symbol as $\alpha \in \Sigma$. Now if L is regular then there exists some DFA that accepts it. Now every DFA is an NFA so let's consider this to be some NFA, N that accepts L . Now we make the following modification,

- (a) In our new NFA labeled M , we create two copies of N . If r_i denote the states of N then p_i and q_i denote the new copies corresponding to r_i such that if δ is our new transition function and δ_0 was that of N then we have $\delta(q_i, k) = \delta_0(r_i, k)$ and $\delta(p_i, k) = \delta_0(r_i, k)$. We also make sure to unmark the start and accept states in M except for that in p (i.e if r_k is an accept state in N then we unmark q_k as an accept state but p_k will still be an accept state)
- (b) We now add a new start state q_α such that $\delta(q_\alpha, \varepsilon) = p_1$ and $\delta(q_\alpha, \alpha) = p_1$ for all $\alpha \in \Sigma$.

(c) Now for each q_i we have $\delta(q_i, \alpha) = p_i$ for every $\alpha \in \Sigma$



In this case the modification we make is as follows,



Now we need to show that M accepts all strings in $\{xay : xy \in L, a \in \Sigma\}$. Consider two arbitrary strings $x, y \in \Sigma^*$ such that $xy \in L$. And take an arbitrary symbol $a \in \Sigma$. Now note that as $x \in L$ the q states will simulate x as in the original *DFA*. Now given the symbol a the NFA can choose to transition to the corresponding state in p , now as the states p are also a perfect simulation of the original *DFA* the string y will also be accepted. Hence, we show that any string of the form xay such that $xy \in L, a \in \Sigma, x, y \in \Sigma^*$ is accepted by M .

(2 points - *Section X*) Let L be a regular language, and let $L^\#$ be the set $\{x \in \Sigma^* \mid \text{for some } y \in L, y \text{ has the same number of 1's as } x\}$. Prove that, if L is regular, $L^\#$ is regular.

Answer: If L is regular, there is a *DFA* for it, and consider this to be the *NFA* of L called N . Now for each state $q \in Q$ of N , we modify δ such that $\delta(q, a) = q$ where $a \in \Sigma$ and $a \neq 1$ and for any a such that $\delta(q_0, a) = q_1$ where $q_0 \neq q_1$ we add $\delta(q_0, \varepsilon) = q_1$. The point of these two additions is that we allow each state to loop to itself any number of times as long as it's not a 1 as we only care about the counts of 1's. And the second modification allows us to skip transitions that are not 1 as again we don't want to enforce the other structures of L that do not count the 1.

Now assume that x is a string accepted by our new *NFA*. Now running down the computation for this string, at each state we have three choices. To self loop an alphabet other than 1, to ε transition (if it exists) or to take the transition like that of L . Now the first two options do not add or remove from the number of 1s (note that

although it's possible for us to have an ε and 1 transition, the ε transition implies that in the original DFA it was also possible to skip the 1), so the third option would be to follow the transition of L in which case we preserve the count of 1. So we show the computation preserves the count of 1 to that of L and hence the string is in $L^\#$.

Now assume $x \in L^\#$ we need to show that x is a string accepted by our NFA. Now clearly x has the same number of 1's as in some string y in L . Now this means that by adding or removing non-1's from y we can construct x . Let $y_0 \dots y_n$ be our string, then after each y_i we can either add non-1's or for each $y_i \neq 1$ we can replace it with ε . So first note that y is trivially accepted by our NFA as we preserve the DFA of L . So for each y_i our new NFA is in some state q_i , now addition of non-1 states is allowed as we allow self loops for non-1s, and removal of $y_i \neq 1$ is allowed by allow q_i to skip to q_{i+1} . Hence, our NFA is able to simulate any function that transforms $y \in L$ to $x \in L^\#$ and hence we have $x \in L^\#$

2. Regular Expressions ***Both Sections*** (4 points)

For the following parts, you may assume $\Sigma = \{0, 1\}$.

(2 points) Give a regular expression for each of the following languages:

- The set of all strings with a multiple of three number of ones. **Answer:**The regex, $0^*(10^*10^*10^*)^*$ represents all strings with multiple of three number of ones.
- The set of all strings where the number of 0's is equivalent to 2 mod 3.

Answer:We have $1^*01^*0(01^*01^*0|1)^*$

(2 points) Give an equivalent NFA for the following regular expression:

$((11 \cup 01^*1 \cup 01)0)^*(11)^*$

Answer:

3. Pumping Lemma (4 points)

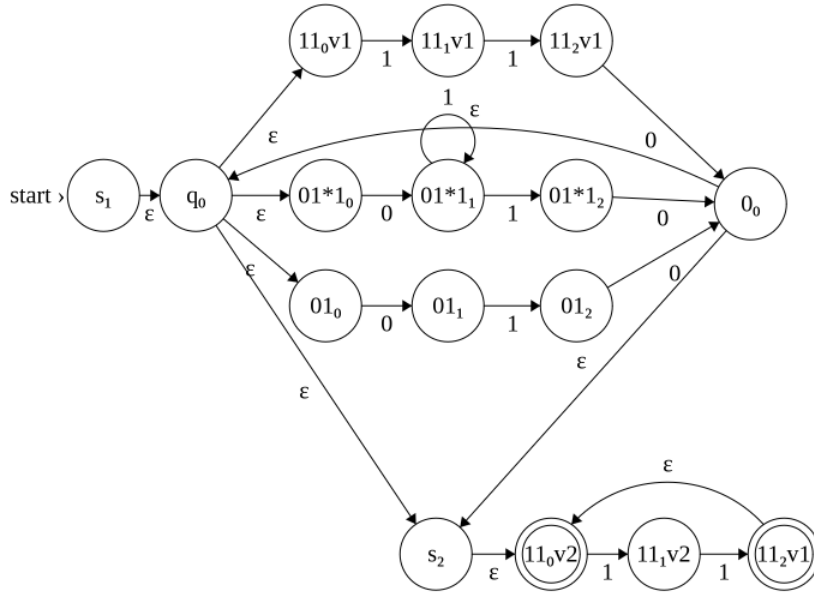
(Section A) Prove that $L_1 = \{a^i b^j \mid |i - j| \text{ is prime}\}$ is not regular.

Answer:

(Section X) Prove that $L_2 = \{a^i b^j \mid i, j \text{ are relatively prime}\}$ is not regular.

Answer:We have $a^i b^j$ where i, j are coprime. Assume that the language is regular with pumping length p . So consider the string $a^q b^{q+1}$ where q is a prime number greater than p then this can be written as $s = xyz$ where for any $i > 0$ we have $xy^i z$ a element of our language. Now note we have three options for y ,

- y contains both a, b in this case note that we would end up with a's and b's interleaved and hence would not be in our language.
- y consists of only a 's. Note that we have the condition that $|xz| \leq p$, but if y consists of only a 's and because $z = b^{q+1}$ and $q + 1 > p$ we have it that $|xz| > p$ a contradiction.



- (c) So the only option is if y consists of only b 's. Note that in this case we have $a^q b^k b^l$ such that $k + l = q + 1$. Now note that if we have y^i for any i then we have $ki + l$ b 's. However, notice in the equation $ki \equiv -l \pmod{q}$ as q is prime this means that $(k, q) = 1$ and hence k has an inverse modulo q . Hence, we have $i \equiv -lk^{-1} \pmod{q}$ an equivalently an i such that $q \mid ki + l$ or $(q, ki + l) > 1$. But this implies that our string xy^iz is NOT a string in our language as the powers of a and b share a common factor, namely q in this case. A contradiction. Hence, this case also doesn't work.

4. Pumping Lemma Adjacent ***Section A only*** (4 points)

A *minimal* DFA D for a language L is a DFA such that any DFA for L has at least as many states as D . Complete the steps below to prove the following claim:

Claim: For any positive integer $n \geq 3$, there is a language whose minimal DFA contains n states.

Proof: For some $k \in \mathbb{Z}$, let $L_k = \{a^k\}$, a set with one element. Now, suppose D is a minimal DFA for L_k . You will prove that D has at least $k + 2$ states. Let $\{q_i\}_{i=0}^k = q_0, \dots, q_k$ be the sequence of states (not necessarily distinct!) that D follows when reading a^k .

(1 point) Explain why $\{q_i\}_{i=0}^k$ cannot contain any repeated states (hint: the proof of the pumping lemma), and conclude that D has at least $k + 1$ states.

Answer:

(1 point) Now consider the state that D ends in upon reading the string a^{k+1} , and argue that D must have at least $k + 2$ states.

Answer:

(1 point) Demonstrate that D has exactly $k+2$ states by giving an explicit construction of D .

Answer:

(1 point) Complete the conclusion of this proof: “Therefore, we have given a constructive proof of the original claim. For any $n \in \mathbb{Z}, n \geq 3$, the language _____ has a minimal DFA with n states.”

Hint: Your answer should be something, like L_{3n} , in terms of n .

Answer:

5. **Polynomial Length** ***Section X only*** (4 points)

Given $f(n)$, a polynomial with non-negative integral coefficients, let $L_f = \{1^{f(n)} \mid n \in \mathbb{N}\}$.

- For exactly which $f(n)$ is L_f regular? Prove your answer.

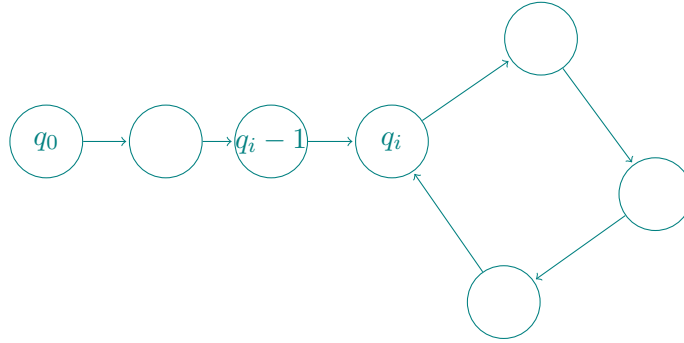
Answer: We know that our alphabet consists of just 1's and that we have a finite automata. First let's analyze all the possible automatas that we can make under these conditions. As we only have one alphabet, each state has only one arrow going away from it or towards it. So we have two cases,

- (a) We have no true loops in our automata. In this case this must mean there is some state that loops to itself.
- (b) Second case is we have only ONE loop. Note that we CANNOT have more than one loop. Assume we have two loops for instance, then there must exist a state that has two transitions for the same alphabet (in this case 1) as we would need one to continue a loop and one to get one of the first loop. However, if we only have ONE loop, we do not need a transition to escape the loop.

Now in both cases if all the accepting states are BEFORE the start of the first state in the loop, then this means that $f(n)$ has a finite range. So clearly all polynomials with finite range will have L_f as a regular language. The DFA in this case is trivial. If $f : \mathbb{N} \rightarrow \mathbb{N}$ with range $A \subset \mathbb{N}$, then we define $k = \max\{A\} + 2$ states, $q_0 \dots q_{k-1}$ where q_i is accepting if $i \in A$ and our transition is $\delta(q_i, 1) = q_{i+1}$ for $i < k$ and for $i = k$ we have $\delta(q_k, 1) = q_k$

Now consider the case where there exists an accepting state a part of the loop. In such a case first we have a finite number of constants i.e. The accepting states before the loop, and for the accepting states within the loop. Note that if a loop exists, there is a state in the loop that has two transitions towards it, one from outside the loop, one from inside. Let $q_0, \dots, q_{i-1}$ be the states outside the loop

and q_i be the start of the loop (and hence the end of the loop as well, we have a transition towards it). Note something like this,



Now let k be the length of the loop, i.e counting the states starting from from q_i to the state transitioning towards q_i , call q_{i+k-1} . In the case of self loop we have $k = 1$. Now for all accepting states in the loop we have q_j such that $i \leq j \leq i + k - 1$. Now our function f must be so that the range of the function is of the form $A \cup B$ where $A = \{k : q_k \text{ is accepting and } k < i\}$ and $B = \{x + i : x \equiv j - k \pmod{k} \forall j \text{ where } q_j \text{ is accepting and } j \geq i\}$. (note that this last condition of f technically also contains the condition that we noted above where all the accepting states are before the first looping state). And more generally we define A such that $|A| = i$ i.e is finite and $B = \{x + i : x \equiv r \pmod{k} \text{ where for some } r \in S \text{ where } S \subset \{0, \dots, k - 1\} \text{ and } k \in \mathbb{N}\}$

We prove that f is regular by constructing the DFA as noted above.

- Prove that for all other $f(n)$, L_f is not regular.

Answer: Clearly every possible DFA we can construct maps onto a function as defined above, which necessarily means that a function not of the form does not have a DFA for it.